

14.1 Need for Generics in Java

Generics in Java were introduced in Java 5 (J2SE 5) to ensure type safety when working with objects. This feature enhances code stability by catching potential errors during compile time, rather than at runtime. Prior to the introduction of generics, collections were non-generic, meaning they could store objects of any type, which posed several issues. Generics, however, enforce storing a specific type of objects in collections, thereby making the code more reliable.

👉 Why Do We Need Generics?

Before generics, arrays were commonly used for storing large datasets. While arrays allow easy storage and retrieval, they require the data type to be specified beforehand. This approach provides type safety by ensuring that only the specified data type can be stored. For example, if an array is defined to hold String values, any attempt to store data of a different type, such as an Integer, will result in a compile-time error.

Let's understand this with an example of storing student names:

👉 Example 1: Storing Data Using an Array

```
public class Demo {
    public static void main(String[] args) {
        String[] names = new String[5];
        names[0] = "Navin";
        names[1] = "Kiran";
        names[2] = "Harsh";
        //...
        names[3] = 10; // This line will cause a compile-time
error

        // Trying to retrieve values
        String name1 = names[0];
        String name2 = names[3]; // This line will also cause
a compile-time error

        System.out.println(name2);
    }
}
```

In the above example, trying to store a number (10) in a String array causes a compile-time error because the array is designed to hold only homogeneous data (String in this case).

👉 Issues with Collections Before Generics

Collections such as ArrayList can store heterogeneous data types. For example, an ArrayList can store a mix of String and Integer values. This flexibility, however, can cause problems during data retrieval, as type mismatches may lead to runtime errors. Here's an example to illustrate this issue:

```
ArrayList list = new ArrayList();
list.add("Ronit"); // Adding a String
list.add("Romil"); // Adding another String
list.add("Rakshit"); // Adding another String

// Mistakenly adding an Integer to the list
list.add(10);

// Trying to retrieve and cast values to String
String n1 = (String) list.get(0);
String n2 = (String) list.get(3); // This line causes a
runtime error
System.out.println(n1 + ":" + n2);
```

Compile Time Error:

```
Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer cann
at MyClass.main(MyClass.java:27)
```

In this case, the program would compile without any issues. However, it will throw a ClassCastException at runtime when trying to cast the integer value to a String. This kind of error can lead to program crashes, making the code unreliable.

👉 How Generics Address These Issues

To resolve these problems, generics were introduced in Java 5, providing type safety and eliminating the need for typecasting. When using generics, we specify the type of objects the collection can hold, ensuring that only the correct data type is stored.

Syntax:

```
ArrayList<String> list = new ArrayList<String>();
```

👉 Advantages of Using Generics

1. Type Safety:

- With generics, collections can hold only a single type of object, and trying to add a different type will result in a compile-time error.
- **Without Generics:**

```
List list = new ArrayList();  
list.add(10); // Adding an integer  
list.add("10"); // Adding a string
```

- **With Generics:**

```
List<Integer> list = new ArrayList<Integer>();  
list.add(10);  
list.add("10"); // Compile-time error
```

2. No Need for Typecasting:

- Before generics, objects had to be typecasted explicitly, which increased the risk of runtime errors.
- **Before Generics:**

```
List list = new ArrayList();  
list.add("hello");  
String s = (String) list.get(0); // Typecasting  
required
```

- **After Generics:**

```
List<String> list = new ArrayList<String>();  
list.add("hello");  
String s = list.get(0); // No typecasting needed
```

3. Compile-Time Checking:

- Generics allow errors to be detected at compile time, reducing the chances of runtime errors and making the code more robust.

Conclusion

Java generics provide a powerful way to create type-safe and stable programs. By enforcing the use of specific data types in collections, generics help catch errors at compile time, reduce the need for typecasting, and improve code readability.